Multi-Modal Vehicle Tracking and Counting System Using Optical Flow and Deep Learning

Group [NUMBER] Arshia Rahim, Ansugan Subramaniam, Wajeehul Hassan

November 20, 2025

1. Project Title, Group Information, and Member Roles

Project Title: Multi-Modal Vehicle Tracking and Counting System Using Optical Flow and Deep Learning
Group Number: [NUMBER]
Group Members and Roles:

Arshia Rahim	Detection & Tracking (YOLOv8, Lucas-Kanade, feature detection)
Ansugan Subramaniam	Data Association & State Estimation (Kalman filter, Hungarian algorithm, track management)
Wajeehul Hassan	System Integration & UI (Streamlit interface, visualization, optimization)

2. Introduction: Problem Statement and Chosen Algorithms

Problem Statement: Accurate vehicle counting in traffic videos faces challenges: occlusions, similar appearances, varying lighting, and real-time requirements. We combine classical CV (optical flow) with deep learning (YOLOv8) for robust tracking.

Chosen Algorithms and Models:

Lucas-Kanade Optical Flow	Sparse feature tracking: $\nabla I \cdot \mathbf{v} + I_t = 0$
YOLOv8 Detection	Single-stage deep learning detector for real-time vehicle detection
Kalman Filter	State estimation ($\mathbf{x} = [x, y, v_x, v_y]^T$) for occlusion handling
Hungarian Algorithm	Optimal data association using IoU + appearance features
Perspective Transform	Homography matrix for bird's-eye view transformation

3. Project Objectives

Primary	Achieve $> 95\%$ counting accuracy on traffic video datasets
Technical	Real-time processing (> 15 FPS), robust occlusion handling ($> 80\%$ persistence), multi-vehicle support
Usability	Develop intuitive web interface with real-time parameter adjustment
Research	Compare optical flow vs. deep learning tracking approaches

4. Experimental Methodology

Datasets: Custom traffic video dataset (2+ videos) with varying densities, lighting, camera angles. Manual annotation for ground truth.

Evaluation Metrics: Counting accuracy ($\frac{\text{True Count}}{\text{Ground Truth}} \times 100\%$), tracking precision (MOTA), FPS, false positive rate.

Experimental Setup: Python 3.8+, OpenCV 4.8+, Ultralytics YOLO, Streamlit. Hardware: CPU with multi-core support (recommended) or GPU with CUDA support for faster YOLO inference. Baseline: Optical Flow vs. YOLOv8. Parameters: confidence (0.3–0.7), min box size (20–50px).

5. Current Status

6. Timeline and Projected Milestones

Deliverables: Functional web application, evaluation report with metrics, documented source code, final presentation.

Completed	Lucas-Kanade tracker, YOLOv8 integration, Kalman filter, data association (IoU + appearance), vehicle counter (line/polygon ROI), perspective transform, CLI interface
In Progress	Counting accuracy improvements, Streamlit web interface, performance optimization ($>$ 15 FPS), enhanced occlusion handling
Challenges	Track ID switching in high-density traffic, similar vehicle appearance, adaptive thresholding, memory optimization

	Arshia Rahim	**Ansugan Subramaniam**	**Wajeehul Hassan**
Week 1–2	Fix counting accuracy, improve YOLOv8 detection	Enhance Kalman filter, improve track persistence	Adaptive thresholding, initial UI framework
Week 3	Optimize detection parameters, multi-scale testing	Data association improvements, track management	Complete Streamlit interface (upload, ROI, parameters)
Week 4	Testing detection accuracy on multiple datasets	Benchmarking tracking performance, accuracy evaluation	UI testing, integration testing, visualization
Week 5	Performance optimization (GPU acceleration)	Code documentation, algorithm documentation	Multi-threading optimization, final UI polish
Week 6	**All:** Final report preparation, presentation slides, project submission		